

Applied Supervised Machine Learning Model for Music Popularity Prediction

Daniel Polanco Castro
Department of Computer Science
Kansas State University
Manhattan, Kansas

Abdulla Mansoor
Department of Business
Kansas State University
Manhattan, Kansas

Abstract— Music popularity prediction is an applied artificial intelligence problem in which structured song features are used to estimate the platform-level success of a track. This project investigates whether Spotify audio and metadata features can be used to predict song popularity through supervised machine learning regression. Using a Spotify dataset from Kaggle, the system preprocesses track-level features, separates the data into training and testing sets, and evaluates multiple regression models, including a naive mean baseline, linear models, decision tree regression, and random forest regression. The project is framed as an applied AI system rather than a reinforcement learning system because the model does not interact with an environment or learn from reward-based feedback. Instead, it learns from labeled examples where each song has known input features and an observed popularity score. Model performance is evaluated using mean absolute error, root mean squared error, and R^2 , with cross-validation used to improve the reliability of model comparison. The evaluation emphasizes comparison against the naive baseline to determine whether more complex models learn meaningful patterns beyond simply predicting average popularity. Error analysis further examines cases where the model underpredicts highly popular songs or overpredicts songs with popular-sounding audio features. Results suggest that supervised regression models can capture useful relationships between audio characteristics and popularity, but their predictive power is limited by missing external factors such as artist reputation, playlist placement, marketing exposure, release timing, and social media trends. This study demonstrates both the usefulness and limitations of audio-feature-based popularity prediction within an applied AI pipeline.

Keywords—*metadata, regression models, decision tree, random forest, mean absolute error, root mean squared error.*

I. INTRODUCTION

This project addresses the problem of predicting song popularity from structured music features using supervised machine learning. Unlike reinforcement learning systems, the proposed system does not involve agents interacting with an environment or learning from reward feedback. Instead, it uses labeled historical data in which each track has known descriptive features and an observed popularity score. Therefore, the task is formulated as a regression problem within applied AI without classification. This moves in line with the selected methodological pillar and category track for the project.

II. BACKGROUND & RELATED WORK

A. Music popularity prediction

Various previous works have proven the link between music and emotion. Research was done on how audio features such as valence, danceability and tempo can generate certain emotions, and has studied the reason why people constantly listen to music as an emotional trigger (Duman et al, 2022). Furthermore, there has been other research on hit song popularity by using audio and human labeled markers that proved the difficulty of finding a hit song (Rompolas et al, 2024). This shows that music popularity prediction is a topic worth further research and development of more models and evaluations.

B. Supervised regression in applied AI

The project embarks on a numerical database for each of its audio-related components, which is the focus of the work. The use of regression allows the machine learning model to learn through its various inputs of numerical audio metadata and pair it with a popularity, the target variable. Therefore, using regression is the most logical choice when focusing purely on audio metadata.

C. Ensemble learning for systematic data

The model for the project is based on the random forest. This method allowed the model to increase its accuracy, reduce overfitting, and handle the larger dataset by creating multiple smaller weaker trees that are averaged using bagging (Breiman,

2001). With a large database of songs and the need to combine the numerical data and target the correct variable, this model proved to be the most effective in increasing accuracy for our conditions and not compromising run-times.

D. Limitations of audio predictions

The project focuses on only audio metadata, excluding the artist, release date, and other categories. This is done to focus on training a reliable model that can later be compared to other models considering no other data was used. This can further the research done in music popularity prediction. Furthermore, simplifying the scope allows for more detailed evaluation of data points, evaluating more deeply the pitfalls of music prediction in machine learning (Pachet and Roy, 2008).

III. METHODOLOGY

A. Dataset description

The dataset has been taken from an open Kaggle database, Spotify Kaggle Database by Solomon Ameh. The unit of the project is each row, which represents one song in the database. The target variable is the popularity of the song, and our inputs are the audio data, which are: danceability, energy, loudness, speechiness, acousticness, instrumentality, liveness, valence, tempo, duration. Each unit is subdivided by the genre of each song, further splitting the dataset.

B. Preprocessing the data

The model was cleaned through several filters. First, the removal of units with missing values to not deal with any incomplete data points. It was checked if the dataset had any duplicates, and any irrelevant columns, such as artist, release date, playlist name, and others were discarded from the curated set. Furthermore, since the original dataset had a global scaling of popularity for each song, but the project is using random forest, there is no feature scaling done. Then the data was split by genre, and the project selects the 6 genres with most songs in the database. This is done to simplify project abstraction and avoid crowding models with too many variables.

C. Task taxonomy

The proposed system belongs to the category of applied artificial intelligence systems using supervised machine learning. It is not a reinforcement learning system because there is no sequential decision-making agent, reward function, or environment. It is also not an unsupervised clustering system because the model is trained with labeled popularity values. Since the output variable is numerical, the prediction task is modeled as regression rather than classification.

D. Baseline model

The project uses a simple baseline model which is used for the regression, $y = y_{train}$. Every song receives the same prediction: the average popularity of the training set. The naive baseline represents the minimum standard that any machine learning model should outperform. If a complex model cannot beat the average-popularity predictor, then the model has not learned about useful relationships between features and popularity.

E. Baseline model design

There were various models reviewed to use in the project. The first was linear regression, proving a simple linear weighted combination between the audio features of a song. It is a transparent model, easy to implement, however as described in the limitations, if the combination of elements is non-linear or affected by unknown variables then the model collapses. Then single regression trees were reviewed, which proved a countermeasure to non-linear combinations of elements, but they are prone to overfitting, especially with the size and depth they grew to be. Random forest was chosen because it is well suited for tabular data, handles nonlinear relationships, captures feature interactions, and reduces the overfitting tendency of single decision trees by averaging predictions across many randomized trees. Breiman's original random forest formulation emphasizes that forests combine multiple tree predictors and can be applied to regression tasks (Breiman, 2001). This model is used by six genre models, not six different algorithms.

F. Hyperparameter design

The random forest model is configured with a fixed random seed to ensure reproducibility. The number of estimators controls the number of trees in the ensemble, while maximum depth and minimum leaf size regulate model complexity (Breiman, 2001). These hyperparameters are important because overly deep trees may memorize training data, while overly restricted trees may underfit. The model is best prepared to predict song popularity under non-linear conditions.

IV. EXPERIMENT DESIGN & RESULTS

The evaluation process was designed to determine whether the proposed supervised regression models learned meaningful relationships between Spotify track features and song popularity. Because the target variable, popularity, is numerical, the project was evaluated as a regression task rather than as a classification or reinforcement learning problem. The evaluation focused on three main goals: first, to measure how accurately each model predicted unseen popularity scores; second, to compare each model against a naive baseline; and third, to identify systematic failure modes in the model's predictions.

To evaluate generalization performance, the dataset was divided into separate training and testing subsets. The training set was used to fit the models, while the testing set was held for final evaluation. This separation is important because evaluating a model on the same data used during training can produce overly optimistic results and may fail to reveal overfitting. The test set therefore served as an approximation of how the model would perform on new, unseen songs.

A. Experimental evaluation design

An 80/20 train-test split was used, where 80% of the available data was assigned to training and 20% was reserved for testing. The split was performed using a fixed random seed to make the experiment reproducible. For the genre-specific modeling approach, the same evaluation logic was applied within each genre subset. In other words, each genre model was

trained only on the training portion of that genre’s data and evaluated on a separate testing portion from the same genre.

In addition to the final train-test evaluation, cross-validation was used during model comparison. Cross-validation provides a more robust estimate of model performance by training and evaluating the model across multiple folds instead of relying on a single random split. This is useful because a single train-test split may produce results that depend too heavily on which songs happen to appear in the training or testing subset. Scikit-learn’s cross-validation tools support evaluating estimators across multiple train-test splits and allow multiple metrics to be reported during validation (Scikit-learn, 2026). The evaluation units were not used for any training, as to ensure the correct training of the model without using seen data from training

B. Evaluation metrics

Three regression metrics were used to determine the accuracy of the model: Mean Absolute Error, Root Mean Squared Error, and R^2 . These metrics were selected because each captures a different aspect of prediction quality.

Mean Absolute Error, or MAE, measures the average absolute difference between the predicted popularity score and the true popularity score:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - y_{mean\ i}|$$

This mathematical equation allows the evaluation to be kept in the same units as the target variable, therefore giving clear results when comparing the baseline to our model prediction.

Root Mean Squared Error, or RMSE, also measured the prediction error but penalizes larger errors than MAE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - y_{mean\ i})^2}$$

The last evaluation criteria R^2 measure the proportion of variance in the target variable.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

C. Overall results

The results show that the genre-specific models improved over the naive baseline for five of the six genres when evaluated using Mean Absolute Error. The strongest improvement occurred in the electronic genre, where the model reduced MAE from 14.10 to 10.49, an improvement of approximately 25.61%. The pop and Latin models also showed meaningful improvement, reducing MAE by 18.62% and 18.25%, respectively.

However, the rock model performed worse than the naive baseline. Its MAE increased from 10.13 to 11.85, and its test

R^2 score was -0.929. This indicates that, for the rock subset, the trained model generalized poorly and performed worse than simply predicting the average rock-song popularity.

TABLE I. Genre-Specific Test Set Results

Genre	Songs	Baseline MAE	Model MAE	MAE Improvement	Baseline RMSE	Model RMSE	Test R^2
Electronic	567	14.10	10.49	25.61%	17.26	13.23	0.389
Pop	494	23.19	18.88	18.62%	26.61	22.91	0.258
Latin	411	18.45	15.08	18.25%	20.93	18.60	0.073
Hip-Hop	370	14.33	13.18	8.05%	18.67	17.57	0.115
Ambient	358	14.40	12.56	12.75%	17.31	15.86	0.157
Rock	318	10.13	11.85	-17.03%	12.37	15.85	-0.929

Across all genre models, the weighted average baseline MAE was approximately **16.17**, while the weighted average model MAE was approximately **13.75**. This represents an overall weighted MAE improvement of approximately **14.99%**. The weighted average RMSE also improved from **19.29** to **17.35**, which represents an approximate **10.07%** reduction in RMSE.

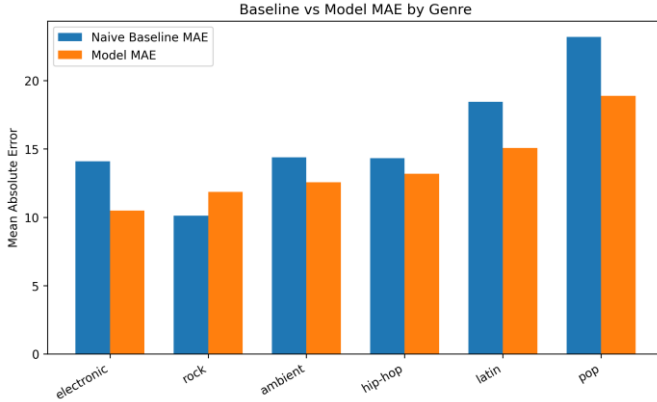
These results suggest that the model learned useful predictive patterns for most genres, but the strength of the prediction varied significantly by genre, otherwise proving the uncertainty of the song prediction.

TABLE II. Cross-Validation Results by Genre

Genre	CV Folds	CV MAE Mean	CV MAE Std. Dev.	CV RMSE Mean	CV RMSE Std. Dev.	CV R^2 Mean	CV R^2 Std. Dev.
Electronic	5	11.22	0.89	14.46	1.14	0.344	0.107
Pop	5	18.24	0.87	23.36	0.71	0.058	0.070
Latin	5	16.08	1.62	19.59	1.57	0.154	0.103
Hip-Hop	5	12.66	1.12	17.00	0.92	0.055	0.106
Ambient	5	12.07	1.24	15.23	0.94	0.045	0.045
Rock	5	12.57	1.00	16.86	1.67	-0.085	0.131

The standard deviations show that most models had moderate variation across folds. Latin and rock had relatively higher RMSE variation, which may indicate greater instability in those genre subsets. This could be caused by smaller sample sizes, uneven popularity distributions, or weaker relationships between the available features and the target variable. The table also shows higher instability of the R^2 values in the rock subgenre, suggesting a bigger spread of the popularity predictions through the songs.

Some other relevant figures that can be drawn from the results of the project:



The figure shows the baseline vs the model MAE by genre. Which shows a general improvement of the MAE with the model compared to the error of the baseline. The model has proven to be a consistent improvement to the regular average and has a better prediction than the baseline.

D. Analysis of results

The best-performing model was the **electronic genre model**. It achieved the lowest model MAE at **10.49**, the strongest test R^2 score at **0.389**, and the highest cross-validation R^2 mean at **0.344**. This suggests that the relationship between Spotify audio features and popularity was strongest for electronic songs in this dataset. One possible explanation is that electronic music may have more consistent relationships between measurable audio characteristics, such as energy, danceability, tempo, and popularity.

The **pop model** had the largest absolute MAE reduction, improving from a baseline MAE of **23.19** to a model MAE of **18.88**. However, its test R^2 score was only **0.258**, and its cross-validation R^2 mean was **0.058**. This suggests that although the pop model improved over the baseline, its performance was not consistently strong across validation folds. Pop music popularity may depend heavily on external factors such as artist reputation, playlist placement, marketing, collaborations, and social media trends.

The **Latin, hip-hop, and ambient models** showed moderate improvement over the baseline. Their MAE values improved, but their R^2 scores remained relatively low. This means the models reduced average prediction error, but they explained only a small portion of the overall variation in popularity. These results suggest that audio features provide some useful signal, but they are not enough to fully predict popularity.

The **rock model** was the main failure case. It performed worse than the baseline on both MAE and RMSE and had a strongly negative test R^2 score. A negative R^2 means that the model's predictions were worse than simply predicting the average

popularity score. This may indicate overfitting, insufficient data, weak feature-target relationships, or greater diversity within the rock genre.

E. Error and failure mode analysis

The evaluation reveals several important failure modes.

First, the model may struggle with **highly popular songs**. Songs that become very popular often do so because of factors that are not included in the dataset, such as artist fame, playlist exposure, marketing campaigns, TikTok trends, radio play, or cultural timing. As a result, the model may underpredict songs that become viral or commercially successful for reasons beyond their audio features.

Second, the model may overpredict songs that have **popular-sounding audio characteristics** but low actual popularity. For example, a song may have high danceability, high energy, or a radio-friendly tempo, but still fail to become popular due to limited exposure or lack of audience reach. This means the model can identify musical features associated with popularity, but it cannot fully determine whether the song reached listeners.

Third, performance varied significantly by genre. The electronic model performed much better than the rock model, suggesting that genre-specific modeling is useful but not equally effective across all categories. Some genres may have clearer relationships between audio features and popularity, while others may be too diverse or too dependent on external cultural factors.

Fourth, the negative performance of the rock model suggests a possible overfitting issue. Since rock had the smallest number of songs in the dataset, with only **318 songs**, the model may not have had enough examples to learn stable patterns. Another possibility is that the rock genre label contains many different substyles, making it difficult for one model to learn a consistent relationship between features and popularity.

Finally, popularity itself is not purely an audio-based property. It is partly social, cultural, and platform dependent. Therefore, even when the model improves the baseline, the relatively low R^2 scores show that a large portion of popularity remains unexplained.

V. CONCLUSION

Overall, the evaluation shows that the project has a meaningful and structured model evaluation process. The models were tested against a naive baseline, evaluated using regression metrics, and validated using five-fold cross-validation. This makes the evaluation stronger than simply reporting one model score.

The results show that the genre-specific models outperformed the baseline in five out of six genres based on MAE. The

strongest result came from the electronic model, which reduced MAE by **25.61%** and achieved the highest test R^2 score of **0.389**. Pop and Latin also showed meaningful improvements over the baseline, while hip-hop and ambient showed smaller but still positive improvements. Rock was the only genre where the model failed to outperform the baseline.

These findings support the conclusion that Spotify audio features can provide useful predictive information for song popularity, especially when models are trained by genre. However, the low-to-moderate R^2 scores also show that audio features alone cannot fully explain popularity. Future improvements should include additional contextual features such as artist popularity, release date, playlist placement, number of streams, social media engagement, and lyrical sentiment.

The evaluation demonstrates that genre-specific supervised regression models can improve over a naive popularity baseline, but the model's limited R^2 performance and genre-dependent failures show that music popularity is influenced by external social, temporal, and platform-level factors beyond the audio features available in the dataset.

A. Future work and expansion

The project shows the promise of comfortably predicting the popularity of a song. For future work, the combination of categorical and numerical data, such as playlists, release date, social media advertising, and other non-audio facts, we can improve the non-linear model further. The future work of the model can help future artists check the probability of releasing a new song that can climb to the top of the charts depending on the multi-variable analysis the model can do. With further work done to the genre analysis, we can break down the project even further to more sub-genres.

VI. REFERENCES

Ameh, S. (n.d.). *Spotify music dataset* [Data set]. Kaggle. Retrieved May 15, 2026, from <https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset/data>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

McLeod, G. D. (2023). *Using Spotify's Web API to examine what audio features contribute to a song's popularity*. Winona State University. <https://openriver.winona.edu/rca/2023/schedule/55/>

Poole, D. L., & Mackworth, A. K. (2023). *Artificial intelligence: Foundations of computational agents* (3rd ed.). Cambridge University Press. <https://doi.org/10.1017/9781009258227>

Saragih, H. S. (2023). Predicting song popularity based on Spotify's audio features: Insights from the Indonesian streaming users. *Cogent Business & Management*, 10(2), Article 2239824. <https://doi.org/10.1080/23311975.2023.2239824>

Scikit-learn developers. (n.d.-a). 3.1. *Cross-validation: Evaluating estimator performance*. Scikit-learn. Retrieved May 15, 2026, from https://scikit-learn.org/stable/modules/cross_validation.html

Scikit-learn developers. (n.d.-b). *DummyRegressor*. Scikit-learn. Retrieved May 15, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyRegressor.html>

Scikit-learn developers. (n.d.-c). *Mean absolute error*. Scikit-learn. Retrieved May 15, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html

Scikit-learn developers. (n.d.-d). *RandomForestRegressor*. Scikit-learn. Retrieved May 15, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

Scikit-learn developers. (n.d.-e). *R^2 score, the coefficient of determination*. Scikit-learn. Retrieved May 15, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html

Scikit-learn developers. (n.d.-f). *Train test split*. Scikit-learn. Retrieved May 15, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

Scikit-learn developers. (n.d.-g). *Model evaluation: Quantifying the quality of predictions*. Scikit-learn. Retrieved May 15, 2026, from https://scikit-learn.org/stable/modules/model_evaluation.html